ENGINEERING PORTFOLIO

최의형

세종대학교 지능기전공학부 무인이동체공학전공 · 2027.02 졸업 예정 · 평점 4.11 / 4.5
Tel 010-4125-3677 | Email dmlgud0777@gmail.com | GitHub github.com/uihyeong

원리를 직접 구현하고, 수치로 검증하는 엔지니어

이론을 코드로 구현하고, 시뮬레이션에서 먼저 검증한 뒤 실물에서 오차를 측정·보정해 완성해 왔습니다. 시스템 설계, 제어, 검증까지 한 사이클을 직접 끝까지 경험했습니다.

01 소개 · 역량

제어의 원리를 수식과 코드로 직접 확인하는 엔지니어입니다. 시뮬레이션에서 검증한 결과를 실물에서 다시 확인하며 그 간극을 좁혀 왔습니다.

직접 구현한 것 해석적 IK 유도·구현, MPC 예측 모델과 QP 솔버, LQR·MPC·CTC 설계와 튜닝, 상태 머신, ROS2 노드 인터페이스
가져다 활용한 것 YOLOv8, EasyOCR, ROS2, Isaac Sim, RealSense SDK, MATLAB/Simulink 등 도구와 학습 모델

학력 · 활동 · 수상

~ 2027.02	**세종대학교 지능기전공학부 무인이동체공학전공** 졸업 예정 **평점평균 4.11 / 4.5** · 127학점 취득
주요 이수 교과	**A+** 동역학 · 동적시스템모델링 · 자동제어 · 로보틱스 · 자율이동체개론및실습 · 기계학습 · 인공지능 · 신호및시스템 · MATLAB프로그래밍 · 디지털신호처리 · Capstone 디자인(산학협력 프로젝트)
2026.07.16	**S.M.A.R.T Tournament 장려상** · 사물인터넷 혁신융합대학사업단 산학연계 프로젝트 경진대회
2026.02 (28h)	**차량용 SW 개발 표준·센서 자율주행 통합 부트캠프** · 현대엔지비 (A-SPICE, ISO 26262, ROS 기반 실습)
2025 하계	**창의학기제 (자기주도창의전공 I, A+)** · 6-DOF 로봇팔 Nuri3s 외란관측기(DOB) 기반 강인제어 (팀 프로젝트, 내 담당: CTC 설계, 제어기 게인 조정·시뮬레이션)
2025.01 ~ 현재	**학부연구생** · 세종대 AI로봇학과 제어 및 자율시스템 연구실 (로봇 동역학·제어 이론 학습, Quanser 1-DOF 비틀림 시스템 제어 연구, 세미나 발표)

자격 · 어학

2026.08	토익스피킹 IM2
2026.03	컴퓨터활용능력 2급
2024.09	ADsP (데이터분석준전문가)
2021.01	자동차운전면허 1종보통

핵심 역량

역량	근거
로봇팔 운동학·제어	OpenMANIPULATOR-X(4-DOF) 해석적 IK를 수식 유도부터 구현, 수 ms 이내 계산으로 실시간 버튼 조작에 사용
인식·센서 통합	RealSense D435 뎁스 + YOLOv8 (UP/DOWN 버튼 mAP50 98.7%), EasyOCR 숫자 판독, HSV 점등 판정
Sim-to-Real 검증	Isaac Sim에서 동작·ROS2 파이프라인 검증 후 실물 이식, 뎁스 기반 Z 오차 실측 보정
ROS2 시스템 통합	토픽 인터페이스·상태 머신 설계, 주행 노드와 연동 (픽업 → 엘리베이터 → 배달 → 회수 미션)
제어 이론	PID·LQR·MPC·CTC 설계, 차량 경로추종 4종 비교, 6-DOF 로봇팔 DOB 적용 시뮬레이션(팀 프로젝트)

기술 스택

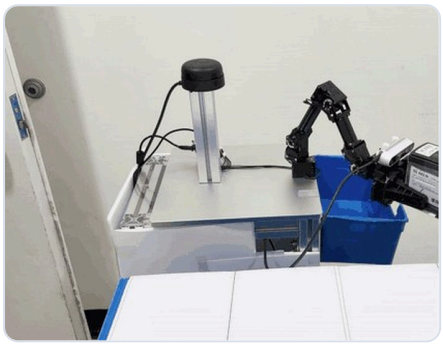
Robotics	ROS2 Humble · Isaac Sim 5.1 · OpenMANIPULATOR-X · 해석적 역기구학(IK) · RealSense D435
AI · Learning	YOLOv8, EasyOCR 기반 인식 · 지도학습(객체 인식 모델 학습, 접촉 분류)
Control	MATLAB / Simulink · PID · LQR · MPC · CTC · DOB
Dev	Python · Git · C

02 캡스톤 · APS (Automated Parcel System)

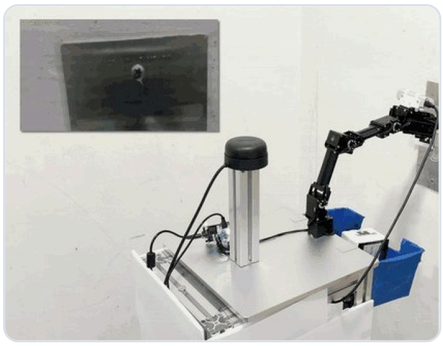
야간 배송기사를 돕는 모바일 매니퓰레이터 · 5인 팀 · 4학년 캡스톤 디자인 · S.M.A.R.T Tournament 장려상

문제. 택배기사가 아파트 입구에 둔 물건을 로봇이 스스로 엘리베이터를 타고 각 세대까지 배송·회수한다. 정형화된 컨베이어가 아니라 매번 달라지는 물건과 실제 건물이라는 **비정형 환경**에서 동작해야 한다.

결과. 픽업 → 버튼 조작 → 층간 이동 → 배달 → 회수까지 **End-to-End 시연 성공**. 실제 건물(지하 1층 → 5층) 실측 기준, 기사가 직접 엘리베이터를 타고 올라가 전달하는 시간 대비 송장을 찍고 로봇 앞에 두고 가는 시간이 **82% 단축**.



① 픽업 — 박스를 집어 바구니로



② UP 버튼 — 인식 → 누름 → 점등 확인



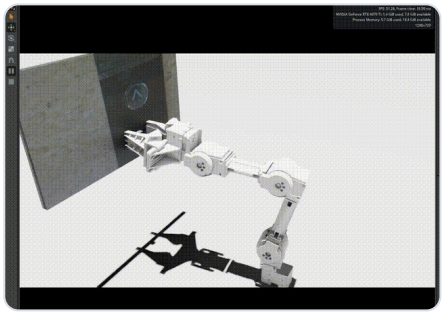
③ 층수 버튼 — 엘리베이터 내부 패널



④ 배달 — 목적지에 내려놓기



⑤ 회수 — 보냉백 손잡이 재거치



⑥ Isaac Sim — 실물 이식 전 동작 검증

실제 건물 촬영 · 시연 영상과 코드: github.com/uihyeong/elevator-button-robot

역할 분담

내 담당 로봇팔 파트 전체 (시뮬레이션 설계 → 실물 구현): 버튼 인식·조작, 픽업/배달/회수 모션, 호수 인식, 접촉 감지, 미션 시퀀싱

팀원 담당 Scout Mini 자율주행 이동·정렬, 앱/서버(층수 입력 HTTPS API) · 각 파트는 ROS2 토픽으로 느슨하게 결합

로봇팔 파이프라인

인식	▶	좌표 산출	▶	해석적 IK	▶	구동	▶	시퀀서
RealSense D435 YOLOv8 · EasyOCR		뎁스 → 목표 X, Y, Z		좌표 → 관절각 (수 ms 이내)		Dynamixel 관절 제어		/robot_status 상태 머신

개발 방식: 시뮬레이션 우선

Isaac Sim에 로봇팔 모델을 불러와 해석적 IK, 50 Hz PID 관절 제어 노드, 인식 파이프라인을 먼저 검증한 뒤 실물에 적용했습니다. 위험과 시행착오를 가상 환경에서 줄이고, 실물에서는 **시뮬레이션과의 간극**에만 집중해 풀었습니다. 상태 머신과 실패 처리, 시뮬레이션-실물 오차 보정의 상세는 **4~5쪽**에서 다룹니다.

03 핵심 기술 ① 해석적 IK 직접 구현

수평 접근 자세를 직접 통제하기 위한 설계

상황	엘리베이터 버튼은 벽에 붙어 있어 팔을 **수평으로 곧게 뻗어 정면에서** 눌러야 한다. 인식 → 좌표 계산 → 접근 → 접촉이 짧은 주기로 반복되는 실시간 태스크다.
판단	버튼을 누르려면 **수평 접근 자세**를 통제해야 했다. 시뮬레이션에서 MoveIt2 버전과 직접 IK 버전을 모두 구현해 본 뒤, 이 자세를 수식으로 직접 구속할 수 있는 직접 IK를 택했다.
행동	4-DOF 구조에서는 수평 접근까지 포함한 닫힌 형태 해가 존재한다고 보고 **수식을 직접 유도·구현**. elbow-up / elbow-down 두 해를 계산하고 관절 한계를 통과하는 해를 선택했다.
결과	좌표 입력 → 관절각 계산이 **수 ms 이내**로 끝나 제어 루프에서 직접 사용할 수 있었다. 오차가 생겼을 때 어느 단계를 볼지 즉시 판단할 수 있었다.

유도 과정 요약

링크 파라미터 L_1=0.0595 · $L_2=\sqrt{(0.024^2+0.128^2)}\approx0.1302$ · L_3=0.124 · L_4=0.126 m

오프셋각 $\alpha = \text{atan2}(0.128, 0.024) \approx 79.4°$ (link2 수평 기준각)

① $\theta_1 = \text{atan2}(Y, X)$ (홈 +π 부근 좌측 버튼은 θ_1+2π 후보도 탐색)

② 손목 위치: $r_w = \sqrt{(X^2+Y^2)} - L_4$, $d_z = Z - L_1$, $D = \sqrt{(r_w^2+d_z^2)}$

③ $\cos\psi = (D^2 - L_2^2 - L_3^2) / (2\,L_2\,L_3)$ (코사인 법칙, ±ψ 두 해)

④ $\gamma = \text{atan2}(L_3 \sin\psi, L_2 + L_3\cos\psi)$

⑤ $\theta_2 = \alpha - (\text{atan2}(d_z, r_w) - \gamma)$, $\theta_3 = -\psi - \alpha$

⑥ **$\theta_4 = -(\theta_2 + \theta_3)$** : 엔드이펙터 수평 접근 구속

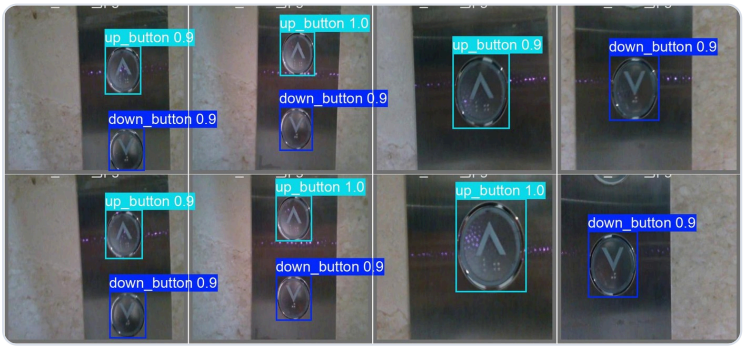
안전장치

- 두 해(elbow-up / elbow-down) 중 **관절 한계를 통과하는 해**만 선택, 해가 없으면 동작하지 않고 실패 처리
- 단일 관절 **이동량 상한 4.5 rad** 초과 시 동작 차단 — 정상 동작의 최대 이동(홈 ↔ 버튼, 약 π rad)보다 크고 한 바퀴 회전(2π)보다 작게 잡아 위험한 회전만 걸러냄
- /joint_states를 받지 못하면 현재 자세를 알 수 없으므로 이동을 중단

04 핵심 기술 ② 인식 · 접촉 감지

98.7%
UP/DOWN 버튼 인식 mAP50 (YOLOv8)

0.812
접촉 감지 F1 (SVM, 윈도우 단위 5-fold CV)



YOLOv8 검증 결과 (up_button / down_button)

- **인식** UP/DOWN은 YOLOv8, 층수 버튼은 YOLO-seg 후 EasyOCR로 숫자 판독
- **점등 확인** UP은 HSV 녹색 비율, DOWN은 밝기(V) 기준으로 점등/소등을 판정해 엘리베이터 도착을 자동 판단
- **접촉 감지** FSR406 + 관절 effort 데이터(19,664개 슬라이딩 윈도우)로 SVM(RBF) 학습, 정지 중 접촉이 감지되면 자동 후퇴. 연속 2윈도우 × 확률 0.70 이상일 때만 확정
- **평가의 한계** F1은 겹치는 윈도우가 분할에 섞인 5-fold CV 값이라 실제 성능은 이보다 낮을 수 있음. 주행 세션 단위로 분할한 재검증이 남은 과제

05 핵심 기술 ③ 시뮬레이션과 실물의 간극

뎁스 오차 실측 보정 · 실패 처리 설계 · 노드 간 동기화

Case A. 뎁스 기반 목표 좌표의 체계적 오차

상황	Isaac Sim에서는 버튼 인식과 도달 궤적이 문제없이 검증됐다.
문제	실물에서는 RealSense 뎁스 기준 목표 Z가 실제 버튼 위치보다 **항상 높게** 계산돼 버튼을 안정적으로 누르지 못했다.
행동	접촉 시도를 반복하며 목표 Z와 실제 접촉 지점의 차이를 **실측**하고, 목표 Z 계산에 **Z − 0.031 m** 보정을 적용했다 (카메라 마운트 TF의 z=0.062가 실제보다 높게 반영되는 오차).
결과	버튼 조작이 안정돼 End-to-End 시연을 완수했다.

왜 상수 보정인가. 카메라 마운트가 고정되어 있었고 오차가 한 방향(실제보다 높게)으로 일정해, 실측으로 구한 **−0.031 m** 하나로 충분하다고 판단해 적용했습니다.

한계와 다음 과제. 카메라 설치 위치가 바뀌면 다시 실측해야 하는 경험적 보정입니다. 핸드-아이 캘리브레이션으로 보정을 체계화하고, 그 위에 데이터·학습 기반 보정을 얹는 것이 다음 과제입니다.

Case B. 실패 처리 설계 — 상태 머신

IDLE /target_floor 수신 ▶ **UP/DOWN** YOLO 인식 → 누름 ▶ **WAIT** 점등 확인 후 소등 = 도착 ▶ **NUMBER** 층수 버튼 인식 → 누름 ▶ **DONE** 홈 복귀

- **UP/DOWN 버튼** 실패·미점등 시 재시도, 연속 3회 실패하면 `NEED_REPOSITION`을 발행해 주행 측에 재정렬을 요청
- **타임아웃** 점등 확인 대기 5~10초, 소등(도착) 대기 60초. 60초를 넘기면 경고 후 도착 확인 없이 다음 단계로 진행하므로, 도착 전에 층수 버튼 단계로 넘어갈 수 있는 위험이 있습니다.
- **알려진 한계** 엘리베이터 안의 층수 버튼 재시도에는 횟수 상한이 없고, `/elevator_ready` 대기에도 타임아웃이 없어 신호가 끊기면 무기한 대기합니다. 재시도 상한과 타임아웃 후 `FAILED` 발행이 개선 과제입니다.

Case C. 로봇팔 ↔ 주행 노드 동기화

문제	각 파트는 단독으로 정상 동작했지만 **연동하는 순간에만** 주행이 끝난 직후 팔이 먼저 움직이는 오류가 반복됐다.
원인	이동 완료 신호의 발행 시점이 팔 노드가 기대하는 시점과 어긋나, 로봇이 완전히 정지하기 전에 팔이 동작했다. (토픽 이름 불일치는 부차적 원인)
행동	팀원과 인터페이스를 재정의해 **정지 완료 신호 방식**으로 바꿨다. 주행 노드는 완전히 정지한 뒤에만 단계별 완료 신호(`/elevator_ready`, `/aligned_ready`)를 발행하고, 팔 노드는 대응하는 대기 상태(`WAIT`, `WAITING_ALIGN`)에서만 신호를 수락하며 그 외 상태에서는 무시한다.
배움	파트별 완성도만큼 **경계의 신호 순서와 상태를 사전에 합의**하는 일이 전체 시스템의 완성도를 좌우한다.

토픽	방향	의미
/elevator_ready	주행 → 팔	엘리베이터 탑승 후 정지 완료 → 숫자 버튼 단계 시작 (`WAIT` 상태에서만 수락)
/aligned_ready	주행 → 팔	호실 앞 정렬·정지 완료 → 배달 시작 (`WAITING_ALIGN` 상태에서만 수락)
/robot_status · /room_number	팔 → 주행	MOVING, UPDOWN_PRESSED, ELEVATOR_ARRIVED, NUMBER_PRESSED, FAILED, NEED_REPOSITION / 인식된 호수

※ `WAITING_ALIGN`은 배달 단계 노드의 대기 상태이고, `DELIVERY_DONE`은 배달 완료 시 같은 노드가 `/robot_status`로 발행하는 값이다.

06 경로추종 제어 ① 설계

Pure Pursuit · Stanley · LQR · MPC 횡방향 경로추종 비교 · 자율이동체개론및실습 (2025 Fall) · MATLAB/Simulink

질문. 고속(60 km/h) 선회에서는 원심력에 의한 슬립과 관성이 지배적이다. **차량을 기구학 모델로 볼 때와 동역학 모델로 볼 때 제어 성능이 얼마나 달라지는가?** 4개 제어기를 동일한 경로·속도 조건에서 직접 구현해 비교했다. 초기 오차는 0으로 고정해 곡률 변화와 원심력에 대한 추종 성능만 비교한다.

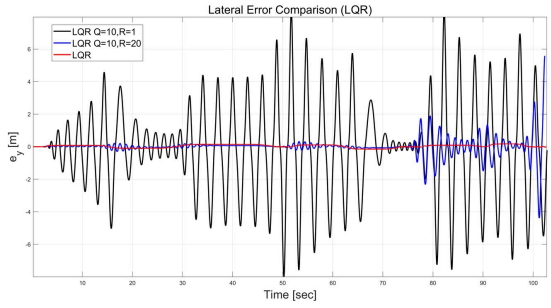
제어기	차량 모델	핵심 원리	설정 · 튜닝 수준
Pure Pursuit	Kinematic	전방 주시점을 향한 원호 추종	속도 의존 $L_d(5+0.3v)$, 속도별 게인, 동적 FF 적용
Stanley	Kinematic	횡방향 + 헤딩 오차 동시 보정	초기 게인(K_ψ=2.5, k=0.5)과 곡률 FF만 적용, 별도 튜닝 없음
LQR	2-DOF Dynamic Bicycle	리카티 방정식으로 최적 게인 + 곡률 피드포워드	Q=diag(2, 1, 2, 1), R=30, 조향 포화 ±0.5 rad
MPC	2-DOF Dynamic Bicycle	예측구간 최적화 + 조향 제약 포함	N_p=20, Q=diag(1, 1, 50, 30), R=5, ±0.5 rad

시뮬레이션 설정: 목표 60 km/h, T_s=0.01 s, 휠베이스 2.7 m. 4종 모두 조향 한계가 걸려 있다.

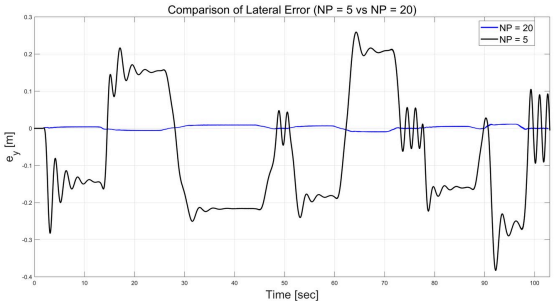
LQR 상태 $x = [e_y, \dot{e_y}, e_\psi, \dot{e_\psi}]^T$
$J = \int (x^T Q x + u^T R u)dt$, $\delta = -Kx + L\kappa$
피드백 + 곡률 피드포워드 (FF 없으면 곡선에서 정상상태 오차)

MPC 상태 $x = [\beta, \gamma, e_\psi, e_y]^T$
$J = \Sigma \| x_k - x_{ref} \|^2_Q + \| u_k \|^2_R$, s.t. $-0.5 \le u_k \le 0.5$ rad
예측 행렬과 QP를 구성하고 **투영 경사하강법 솔버를 직접 구현**

튜닝: 숫자가 아니라 물리적 거동으로 해석



LQR Q/R Q를 크게, R을 작게 둔 경우(검정, Q=10/R=1)는 ±7 m로 발산, R을 올린 경우(파랑)도 잔진동이 남았고 최종 튜닝(빨강)에서 수렴. 발산의 원인은 정밀 분석하지 못했으며, 연속시간 게인을 T_s=0.01 s 이산 제어에 적용한 점과 조향 포화가 영향했을 가능성이 있다(추정).



MPC 예측구간 N_p=5(검정)는 ±0.2~0.4 m로 진동, N_p=20(파랑)은 0 근처로 안정. 계산 부하와의 절충으로 N_p=20을 선정.

구현 메모

- 제어기는 Simulink의 MATLAB Function 블록으로 구현했고, 궤적·횡방향/헤딩 오차·조향각·횡가속도·곡률·종방향 속도를 한 번에 기록하는 대시보드를 구성했다.
- 코드·Simulink 모델·결과 데이터: github.com/uihyeong/path-tracking-control

07 경로추종 제어 ② 결과와 해석

0.0059 m
레이싱 트랙 MPC 횡방향 RMSE

≈17배
튜닝한 LQR(0.0981 m) 대비
(PP 대비 ≈29배)

0.6096
MPC Jerk RMSE [m/s³]
(PP 0.6148와 동급, LQR 0.9875보다 낮음)

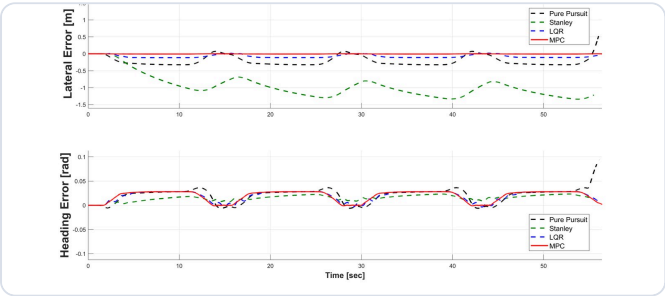
(3) 우회전 경로 · 60 km/h

	Stanley	PP	LQR	MPC
횡방향 RMSE [m]	1.0138	0.1592	0.0883	**0.0054**
최대 횡방향 [m]	1.3406	0.2053	0.1135	**0.0073**
헤딩 RMSE [rad]	0.0164	0.0220	0.0219	0.0224
최대 헤딩 [rad]	0.0234	0.0337	0.0283	0.0282

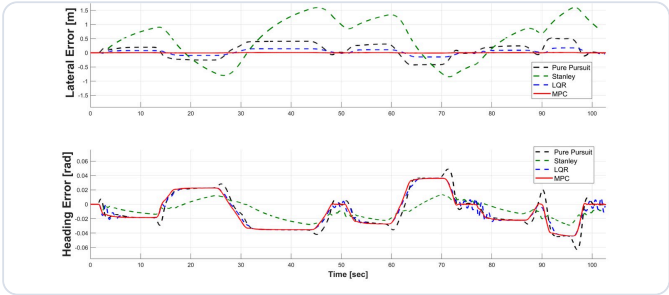
(4) 레이싱 트랙 · 60 km/h

	Stanley	PP	LQR	MPC
횡방향 RMSE [m]	0.8869	0.1724	0.0981	**0.0059**
최대 횡방향 [m]	1.6026	0.3189	0.1732	**0.0117**
헤딩 RMSE [rad]	0.0138	0.0238	0.0239	0.0243
최대 헤딩 [rad]	0.0292	0.0453	0.0444	0.0441
Jerk RMSE [m/s³]	0.6864	0.6148	0.9875	**0.6096**

Stanley는 튜닝 전 초기 게인이다. Jerk는 레이싱 트랙에서만 산출했다. 두 경로 모두 시뮬레이션은 경로 종료까지 진행.



(3) 우회전 · 60 km/h. 급조향 구간에서 Stanley(녹색)만 −1.3 m로 크게 이탈, MPC(빨강)는 0 근처 유지.



(4) 레이싱 트랙 · 60 km/h. 연속 곡률 변화에서 Stanley(녹색)는 매 코너 ±1.6 m로 이탈, LQR(파랑)·MPC(빨강)는 안정 추종.

해석

- **운동학 기반 제어기의 한계와 튜닝 영향.** PP와 Stanley는 슬립·관성을 반영하지 않아 고속 선회에서 코너커팅·이탈이 나타났다. 다만 Stanley는 초기 게인 그대로여서 오차 크기에는 모델 차이뿐 아니라 **튜닝 수준의 영향**도 섞여 있다.
- **LQR의 트레이드오프.** 곡률 피드포워드로 코너커팅은 제거했지만 오차에 민첩하게 반응해 Jerk가 가장 높다(0.9875).
- **MPC.** 오차가 생기기 전에 미리 조향하고 조향 제약을 최적화에 직접 반영해 횡방향 오차가 가장 작다. 승차감은 LQR보다 낫지만 PP와는 동급이다.
- **다면 평가.** 헤딩 RMSE는 4종 모두 약 0.02 rad로 비슷하지만 의미가 다르다. Stanley는 밀려난 채 방향만 정렬된 것이고, LQR/MPC는 위치 보정을 위해 능동적으로 슬립각을 만든 정상 반응이다. 고속에서는 헤딩 오차 단독 평가가 부적절하다.

비교 조건과 한계. 외란과 센서 노이즈가 없는 시뮬레이션이고, LQR·MPC의 예측 모델이 플랜트와 같은 2-DOF 동역학 구조라 MPC에 유리한 조건입니다. 모델 오차, 외란, 센서 노이즈에 대한 강인성은 검증하지 못했고, MPC 풀이 시간도 측정하지 않아 T_s=0.01 s 실시간 구동 가능성은 미검증입니다. 모델 불확실성과 외란에 대한 강인성은 다음 쪽의 Nuri3s 프로젝트에서 로봇팔로 다뤘습니다.

08 강인제어 · 6-DOF 로봇팔 외란관측기(DOB)

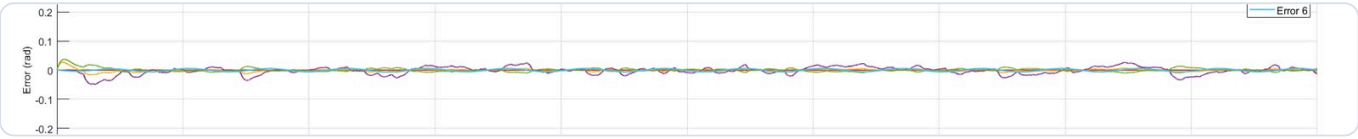
Nuri3s 6-DOF 로봇팔 · 2025 세종대 하계 창의학기제 (팀 프로젝트) · MATLAB/Simulink
github.com/uihyeong/nuri3s-robust-control

내 담당 CTC(계산토크 제어) 설계, 제어기별 게인을 바꿔가며 시뮬레이션해 응답 비교

모델이 정확하면 고전 제어기로도 잘 추종되지만, 실제 로봇의 **모델 불확실성과 외란**이 더해지면 성능이 무너집니다. 이 열화 과정을 단계별로 재현하고 **DOB로 강인성을 회복**하는 흐름을 시뮬레이션으로 검증했습니다.

단계	제어기	조건	결과
0	Open Loop	피드백 없음	오차 발산
1	PD + 중력보상(PD+G)	정확 모델	안정적 목표 추종
2	PD+G	질량·관성 ±10% 불확실 + 외란	정상상태 오차 증가
3	**PD+G + DOB**	**동일 조건**	**추적 오차 ±0.05 rad 이내 수렴**
3'	**CTC + DOB (+Ki)**	**동일 조건**	**외란·불확실성 억제 (정성적 확인, 정량 지표는 산출하지 않음)**

외란: sine 진폭 0.1, 10 rad/s · 실제 모델 vs 추정 모델(URDF의 질량·관성 ±10% 조정)로 불확실성 모사 · DOB 내부 역동역학 계산은 추정 모델 사용



내 담당 · CTC + DOB + Ki (불확실 모델, Gain 7) 6개 관절 위치 오차 [rad]. 축을 ±0.2 rad로 확대해 오차가 ±0.05 rad 안쪽(초기 구간 최대 약 0.04)에 머무는 것이 보인다.



팀 결과 · PD+G + DOB (불확실 모델, Gain 3) 같은 축 범위. 같은 조건에서 DOB를 끈 PD+G와 겹쳐 비교한 그래프는 만들지 못했다(DOB 없는 단계의 결과 데이터 미보관).

게인 (CTC + DOB + Ki, Gain 7)	J1	J2	J3	J4	J5	J6
Kp	900	800	500	300	300	250
Kd	40	45	40	40	30	30
Ki	500	400	300	250	250	150

09 학부연구생 · 제어 이론 학습과 1-DOF 비틀림 시스템 PID / LQR

세종대 AI로봇학과 제어 및 자율시스템 연구실 · 2025.01 ~ 현재 · 연구실 세미나 발표

- **이론 학습** 로봇 동역학과 제어 이론(상태공간 모델링, 최적제어 등)을 자기주도로 학습하고 노션에 정리하며, 연구 실험에 적용
- **대상** Quanser Rotary Servo(SRV02)에 유연 커플링으로 비틀림 부하를 연결한 1-DOF 비틀림 시스템. 서보 위치를 빠르게 움직이면서 부하의 진동을 줄이는 것이 과제
- **모델링** 서보와 부하의 운동방정식을 세워 4개 상태(서보·부하 각도와 각속도)의 상태방정식을 직접 유도하고, MATLAB/Simulink에서 PID와 LQR을 각각 설계
- **튜닝** Q·R 가중치에 따른 응답 특성(응답 속도 ↔ 진동·안정성 트레이드오프)을 비교 분석
- **실물 적용** 설계한 제어기를 실제 장비에 적용해 이론 모델과 실제 거동의 오차를 측정·검증. 수식상 안정적이던 게인이 그대로 통하지 않아 재튜닝 후에야 안정화됐다. 선형 모델이 반영하지 못하는 정지마찰과 커플링의 비선형이 원인으로 보인다. 이 경험이 캡스톤의 시뮬레이션-실물 오차 보정으로 이어졌다.

10 앞으로의 방향

지원 방향: 로봇·자율주행 제어 소프트웨어 — 학습 기반 제어(강화학습·모방학습)를 기존 제어 스택에 안전하게 통합하는 엔지니어로 성장하는 것이 목표

지금까지는 모델 기반 제어와 시뮬레이션-실물 검증에 집중했습니다. 다음 단계는 같은 문제의식(시뮬레이션에서 잘 되는 것이 실물에서도 되는가)을 **학습 기반 방법**으로 풀고, 대상을 **단일 로봇팔에서 휴머노이드처럼 다관절·이동·조작이 결합된 시스템**으로 넓히는 것입니다.

현재 역량 → 확장 방향

영역	지금까지 한 것	확장하고 싶은 것
이동 + 조작	Scout Mini 위 4-DOF 로봇팔로 이동·정렬 후 버튼 조작·배달 (모바일 매니퓰레이터)	양팔·다관절 휴머노이드의 상체 조작, 이동과 조작을 함께 다루는 전신 제어(Whole-body control)
모델 기반 제어	해석적 IK, PID·LQR·MPC, 6-DOF 로봇팔 CTC 설계와 DOB 적용 시뮬레이션	다관절 동역학과 균형·보행 제어 이론 학습, 모델 기반 제어를 학습 정책의 안전 장치로 활용
Sim-to-Real	Isaac Sim 검증 후 실물 이식, 뎁스 오차를 실측해 상수 보정	핸드-아이 캘리브레이션과 도메인 랜덤화·시스템 식별로 보정을 체계화
학습	YOLOv8 인식 모델 학습(지도학습), 접촉 분류	**강화학습**으로 조작·제어 정책 학습, 모방학습 기반 조작 학습

강화학습으로 해보고 싶은 것

솔직한 현황 강화학습을 실물 로봇이나 시뮬레이션 로봇에 적용해 본 경험은 아직 없습니다. 아래는 이미 구현해 둔 기반을 출발점으로 삼는 학습 계획입니다.

STEP 0 · 기준값
IK 소요 시간, 도달 정확도, 버튼 성공률 측정

▶

STEP 1 · 이론
MDP, PPO/SAC, 간단한 환경에서 직접 구현

▶

STEP 2 · 시뮬레이션
Isaac Lab에서 매니퓰레이터 도달 정책 학습

▶

STEP 3 · 비교
해석적 IK와 정량 비교, 학습 보정(잔차) 결합

▶

STEP 4 · 확장
Sim-to-Real, 휴머노이드 상체 조작 과제

왜 이 방향인가

- **같은 질문의 연장.** 캡스톤에서 실측 상수로 맞춘 Z 보정은 실험실 규모의 임시방편이었고, 이를 캘리브레이션과 데이터·학습으로 체계화하는 것이 가장 자연스러운 다음 과제입니다.
- **측정 가능한 출발점.** STEP 0에서 해석적 IK의 소요 시간·도달 정확도·성공률을 기준값으로 먼저 측정해, 학습 기반 방법의 효과를 정량 비교할 수 있게 하겠습니다.
- **고전 제어를 버리지 않기.** LQR·MPC·CTC로 쌓은 안정성 감각을 바탕으로, 학습 정책이 물리적으로 위험한 동작을 하지 않도록 제약과 보정 구조를 설계하고 싶습니다.
- **휴머노이드로의 연결.** 이동체 위 매니퓰레이터, 센서 기반 조작, ROS2 통합 경험은 휴머노이드의 이동·상체 조작 시스템과 구조적으로 맞닿아 있습니다.

Links

GitHub	github.com/uihyeong
캡스톤 로봇팔	github.com/uihyeong/elevator-button-robot (데모 GIF, 코드, YOLO 가중치)
경로추종 제어	github.com/uihyeong/path-tracking-control (Simulink 모델, 결과 데이터)
강인제어	github.com/uihyeong/nuri3s-robust-control
Contact	010-4125-3677 · dmlgud0777@gmail.com